
第 0 章

前置基础：环境、编程与 LaTeX

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

0. 前置基础：环境、编程与 LaTeX（新版讲义）

本页是第 0 章的章首页 (索引)。正文位于 00-01-课程导览与学习路径.md ~ 00-07-综合案例.md。
本课程原有 8 章（第 1~8 章）无需重编号，第 0 章作为“预备周”放在最前。

本章概览

第 0 章解决开课前的三件“小事”，但它们决定了后面 8 周是否顺畅：

1. **统一环境**：每人一套相同的 Python + VS Code + LaTeX 工具链，用脚本自检，避免“我的电脑装不上”；
2. **编程查漏**：你们学过语言程序设计课，本章不重讲语法百科，而是用 10 道自测题找出缺口，针对性强化容器、函数、异常、文件与常用内置函数；
3. **LaTeX 报告**：掌握“文档骨架 + 公式 + 表格 + 插图 + 一键编译”，以便完成课程报告与期末大作业；同时介绍 Overleaf、LoongTeX 等在线编译平台作为备选。

学习目标

学完本章，你应该能够：

1. 按讲义在你自己的电脑上搭好与课程一致的 Python + VS Code + LaTeX 环境，并通过 check_env.py 自检；
2. 用 VS Code 打开项目、运行 .py 脚本与 .ipynb 笔记本，切换解释器和内核；
3. 完成 Python 强化自测，熟练使用容器切片、列表推导式、函数参数、异常处理、文件读写与 zip/enumerate/sorted 等常用工具；
4. 写出第一个中文 LaTeX 文档并编译出 PDF，能插入公式、表格、图片，能用课程自带模板排版一份小报告；
5. 完成综合案例“从成绩单数据到 LaTeX 报告”，跑通“数据 → 统计 → 图表 → 报告”的完整 workflow。

先修要求与运行环境

- 修过一门语言程序设计课（C/C++/Python 均可），会写变量、条件、循环、函数；
- 不要求任何 LaTeX 基础；
- 建议课前用自己的电脑安装好课程统一环境标准（见下文“统一环境标准”），或直接使用机房已装好的环境。

本章目录

小节	文件	内容
01 课程导览与学习路径	00-01-课程导览与学习路径.md	课程定位、8 章地图、学习方法、作业考核
02 统一环境安装与自检	00-02-统一环境安装与自检.md	Miniconda/VSCode/镜像/环境文件/自检脚本
03 VS Code 与 Jupyter 工作流	00-03-VSCode 使用与 Jupyter 工作流.md	扩展、运行、笔记本、终端、调试、快捷键
04 Python 编程快速强化	00-04-Python 编程快速强化.md	自测、容器、函数、异常、文件、常用内置函数
05 LaTeX 基础与编译	00-05-LaTeX 基础与编译.md	在线平台、TeX Live、LaTeX Workshop、模板、常见错误

小节	文件	内容
06 科学计算工作流与代码规范	00-06-科学计算工作流与代码规范.md	目录、notebook、规范、环境复现、Git 入门
07 综合案例	00-07-综合案例.md	成绩单数据 → 统计 → 图表 → LaTeX 报告

数学预备与附录

本章用到的统计/公式背景已集中到附录《数学与算法补充》：

- **C 概率统计基础**：C.1 描述统计（均值/方差/分位数）、C.2 常用分布（正态/二项/泊松）；
- **F 通用速查**：F.1 记号、F.2 常用公式卡、F.4 数值稳定性；
- [打开附录 C](#) | [附录索引](#)

进入成绩统计前先花 5 分钟读 C.1；上机遇到公式用 F.2 速查。

练习与上机入口

- [本章练习 \(exercises/\)](#)：作业题、自测 quiz（自动评分）、参考答案。
- [本章上机 \(lab/\)](#)：环境自检 + Python 热身 + LaTeX 报告，4 学时。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、学习网站、在线编译平台。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

统一环境标准（本章采用）

项	标准
操作系统	Windows 10/11（macOS/Linux 备注，不重点保障）
Python	3.12（兼容 3.10–3.13）
发行版	Miniconda（Anaconda 可作为备选；纯 Python + venv 也可以）
编辑器	VS Code
扩展	Python、Pylance、Jupyter、LaTeX Workshop（可选 GitLens、Rainbow CSV）
LaTeX	TeX Live + ctex + latexmk，中文用 xelatex 编译；在线备选 Overleaf / LoongTeX
镜像	清华 TUNA（conda 与 pip）
路径	全部英文（如 D:\\Scientific_Computing_Class\\course\\）

建议课时

环节	学时	对应内容
讲课	2 学时（第 0 周）	00-01 ~ 00-06 精讲 + 00-07 案例导入
上机	4 学时（第 0 周）	lab/：环境自检 0.5h + Python 热身 2h + LaTeX 报告 1h + quiz 0.5h

若第 0 周课堂挤不出 2 学时，可采用压缩版：00-01/02/03 并入第 1 周上机课前 1h；00-04 课前自测；00-05 在第一次作业前 45min 当场带练。详见 teaching.md。

使用说明

- **学生**：先读 00-01 ~ 00-06 并动手操作 → 跑 lab/（上机课完成）→ 用 exercises/quiz.ipynb 自测 → 按 00-07 完成第一份 LaTeX 报告。
- **教师**：按 teaching.md 的课时表讲解；lab 是上机课主线；quiz 带自动评分可直接回收；LaTeX 部分建议现场演示“保存 → 自动编译 → 看 PDF”。

01 课程导览：学什么、怎么学、怎么考

本节目标：让你在开课前知道这门课的坐标、8 周怎么走、第 0 章负责什么、作业与考核如何完成。

本节目标

- 理解《Python 科学计算》在知识地图中的位置（上游/下游课程）；
- 记住 8 个主题章与每周的学习循环（上课 2h + 上机 4h）；
- 知道第 0 章要做什么、什么时候做；
- 明确三类产出：上机 lab、练习题 quiz、LaTeX 课程报告。

1.1 这门课讲什么

本课程用 **Python 生态** 完成科学计算、数据分析、统计建模与机器学习小项目：

周 章	主题	一句话说明
0	第 0 章 前置基础	环境、编程强化、LaTeX 报告
1	第 1 章 NumPy	数组、向量化、线性代数、拟合
2	第 2 章 SymPy	符号运算、极限/导数/积分、方程求解
3	第 3 章 SciPy	数值积分、优化、插值、假设检验、FFT
4	第 4 章 Pandas	表格数据、清洗、分组、透视、时序
5	第 5 章 Matplotlib	可视化、图窗布局、seaborn 美化
6	第 6 章 NetworkX	图结构、中心性、最短路、社区发现
7	第 7 章 Statsmodels	回归、方差分析、时间序列
8	第 8 章 scikit-learn	预处理、有监督/无监督学习

第 0 章不独立成“知识主题”，而是给后面 8 章**铺路**：统一环境、补齐 Python 弱点、学会 LaTeX 出报告。

1.2 课程怎么组织

- 学时：**48 学时** = 8 周 × (2h 上课 + 4h 上机)；第 0 周为预备周（标准版 2h 上课 + 4h 上机，可在上机课完成）。
- 每周循环：

上课（2h）：精讲本章核心 + 现场演示

↓

上机（4h）：0.5h 回顾 → 3h 完成 lab → 0.5h 当场 quiz/总结

↓

课后：完成 exercises 作业；自读“常见误区”页；预习下一章

- 详细课表见 教学资源/课时安排总表.md。

1.3 第 0 章怎么学（建议顺序）

1. **课前（30–60 min）**：读 00-01、00-02、00-03，把 Miniconda 与 VS Code 装上；
2. **第一次上机（4h）**：跟 lab 完成环境自检 → Python 热身 → LaTeX 报告；

3. **课后 (2–3h)**: 完成 `exercises/quiz.ipynb` 自测; 写一份《学习目标与课程期待》LaTeX 报告 (用课程模板, 练手);
4. **遇到问题**: 先按 00-02 常见问题表自查, 再问同学/助教; 机房网络不行时用在在线 LaTeX 平台 (见 00-05)。

1.4 教材与工具

载体	用途
<code>chapters/00-prep/*.md</code>	唯一内容源 (建议随时对照)
网页版 (GitHub Pages + docsify)	在线阅读、侧边栏导航、搜索
教材 PDF /	每章合订 PDF 与全书 PDF (构建产物)
教材 TeX /	可编译、可手改的 LaTeX 工程 (教师/讲义用)
教学资源/环境配置/	环境文件、自检脚本、VS Code 配置

1.5 作业、考核与提交

1. **每章 quiz.ipynb**: 自动评分 (`_GRADES` 汇总), 当场或课后完成;
2. **每章 assignment.md**: 课后书面/编程题, 含答案要点;
3. **lab/ 完成情况**: 上机课检查; 第 0 章要求提交 环境自检输出 + lab0 报告 PDF;
4. **期末大作业**: 第 8 周布置, 课程结束后 2–3 周内提交, 要求用 LaTeX 排版报告 (这就是为什么第 0 章必须学 LaTeX)。

提交命名规范 (后面 8 章通用):

学号 _ 姓名 _ 第 0 章 _lab0.py
 学号 _ 姓名 _ 第 0 章 _lab0 报告.pdf
 学号 _ 姓名 _ 第 0 章 _env_check.txt

1.6 常见问题

问题	建议
电脑装环境要多久?	正常 30–60 min; 机房统一预装或提供离线包可压缩到 10 min
我能用在线编译器代替本地 LaTeX 吗?	可以 (Overleaf/LoongTeX), 但上课讲的是本地 VS Code 流程, 建议至少会一种
之前 Python 只学过皮毛, 跟得上吗?	第 0 章 00-04 就是为你准备的: 先自测再查漏; 后面章节会反复用
这门课一定要写 LaTeX 吗?	课程报告与期末大作业要求 PDF 报告, LaTeX 是推荐方式; 不会的话本章就是补课

本节小结与思考题

1. 用自己的话说: 为什么科学计算要用 NumPy 而不是普通 Python 列表? (提示: 类型统一、向量化、连续内存)
2. 你目前 VS Code / Python / LaTeX 各处于什么水平 (没装过 / 装过不会用 / 熟练)? 写下三个薄弱点, 第 0 章结束时再回来看。
3. 预告: 第 7 章 Statsmodels 与第 8 章 scikit-learn 与 “数学建模” 有何联系?

02 统一环境安装与自检

本节目标：按统一标准装好 Python (Miniconda)、VS Code、LaTeX，并用 `check_env.py` 一次性验证环境可用。本节是后面所有章节的地基，请务必在第一次上机前完成。

本节目标

- 理解“统一环境”包含哪些组件 (Python/包管理器/编辑器/内核/LaTeX)；
- 在自己电脑上安装 Miniconda、VS Code，并创建课程环境 `scicomp`；
- 学会配置国内镜像 (conda/pip)，加速安装；
- 会用 `environment.yml` 一键重建环境；
- 运行 `check_env.py` 自检并截图/保存输出作为第 0 周作业。

教学方式 (BYOD)：本课程要求学生**自带笔记本电脑**，课堂与上机都优先在个人电脑上操作；机房仅作为应急/检查点。请在第 0 周带上已装好的电脑，遇到安装问题当场解决（同学互助 + 教师巡场），不要依赖机房预装环境。

2.1 环境总览

```
+-----+
| 你的项目: chapters/00-prep/ |
+-----+
| 编辑器: VS Code           |
| Python / Jupyter / LaTeX Workshop |
+-----+
| 内核: ipykernel           |
+-----+
| 环境: Miniconda scicomp   |
| numpy/sympy/scipy/pandas/ |
| matplotlib/networkx/     |
| statsmodels/scikit-learn  |
+-----+
| 系统: Windows 10/11      |
+-----+
```

配图：images/env_stack.png（同结构示意图）。

2.2 安装 Miniconda

为什么推荐 Miniconda 而不是 Anaconda? 两者内核相同；Anaconda 预装大量用不到的工具（约 2–3 GB），Miniconda 只带最小核心（约 150–200 MB），可由课程 `environment.yml` 精确安装所需包。对机房/学生机更友好。

Windows 步骤

1. 打开清华镜像站 Miniconda 目录：[清华 Miniconda 镜像](#)；
2. 下载 **最新版** `Miniconda3-latest-Windows-x86_64.exe`（约 80 MB）；

3. 双击安装: **为当前用户安装**, 勾选 “Add Miniconda3 to my PATH environment variable” (若没勾, 之后从 “Anaconda Prompt” 启动也行);
4. 安装完成后打开 **Anaconda Prompt** (或重启终端), 验证:

```
conda --version
python --version
```

macOS / Linux 备注

- macOS: 下载 Miniconda3-latest-MacOSX-x86_64.sh (Apple Silicon 用 arm64 版本), 运行 bash 文件名;
- Linux: 下载 Miniconda3-latest-Linux-x86_64.sh, 同样执行; Linux (含 CI) 请额外安装中文字体与 xelatex (见 00-05)。

2.3 配置国内镜像 (可选但强烈推荐)

conda 使用清华镜像

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/
conda config --set show_channel_urls yes
```

若提示 channel 冲突, 也可只保留 default 并在安装时加 -c 镜像通道, 例如 [pkgs/main](https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/)。

pip 使用清华镜像

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip config set global.trusted-host pypi.tuna.tsinghua.edu.cn
```

2.4 创建课程统一环境

课程已提供环境定义文件: 教学资源/环境配置/environment.yml。把它复制到你的课程目录 (或直接引用绝对路径), 然后:

```
cd D:\Scientific_Computing_Class\course-dist # 或你的课程目录
conda env create -f environment.yml
conda activate scicomp
```

如果以后想重建 (比如学期中途更新): 先 conda env remove -n scicomp 再 create。若你更喜欢 pip:

```
conda create -n scicomp python=3.12 -y
conda activate scicomp
pip install -r requirements.txt
```

环境文件里包含: numpy、sympy、scipy、pandas、matplotlib、seaborn、networkx、statsmodels、scikit-learn、jupyterlab/ipykernel。第 1~8 章全部够用。

2.5 安装 VS Code 与扩展

安装 VS Code

1. 打开 [链接](<https://code.visualstudio.com/>, 下载) Windows 安装包并安装 (默认选项即可);
2. 安装完成后, 打开命令行确认:

```
code --version
```

安装必需扩展

在 VS Code 扩展市场 (Ctrl+Shift+X) 搜索并安装:

扩展	扩展 ID	用途
Python	ms-python.python	解释器、运行/调试
Pylance	ms-python.pylance	类型检查、补全
Jupyter	ms-toolsai.jupyter	运行.ipynb 笔记本
LaTeX Workshop	james-yu.latex-workshop	LaTeX 编辑/自动编译
Rainbow CSV (可选)	mechatroner.rainbow-csv	方便查看 csv
GitLens (可选)	eamodio.gitlens	代码历史/协作

也可以在命令行一键安装:

```
code --install-extension ms-python.python
code --install-extension ms-python.pylance
code --install-extension ms-toolsai.jupyter
code --install-extension james-yu.latex-workshop
```

导入课程共享配置

打开任意课程目录, 把 教学资源/环境配置/vscode-settings.json 的内容合并进你的 设置 (JSON); 关键项包括:

- python.defaultInterpreterPath: 指向 scicomp 环境的 Python;
- jupyter.notebookFileRoot: 项目根目录;
- latex-workshop.latex.recipes: latexmk -xelatex (中文必须)。

配置片段见 00-03 与 00-05; 直接复制文件即可。

2.6 环境自检

课程提供自检脚本: 教学资源/环境配置/check_env.py。用法:

```
conda activate scicomp
python check_env.py
```

预期输出 (节选):

```
[OK ] Python 3.12.x (scicomp)
[OK ] numpy 2.x / sympy 1.13 / scipy 1.15 / pandas 2.2 ...
[OK ] matplotlib / networkx / statsmodels / scikit-learn
[OK ] xelatex found: C:\texlive\2026\bin\windows\xelatex.exe
[OK ] latexmk found
[WARN] git: 未安装 (可选)
[OK ] 环境自检通过 (0 个必须项失败)
```

提交要求 (第 0 周 lab A 部分): 保存输出为 env_check.txt (或截图), 命名 学号 _ 姓名 _ 第 0 章 _env_check.txt。

2.7 可选：准备课程目录

建议统一目录（全英文路径）：

```
D:\Scientific_Computing_Class\  
+- course-dist\           # 你的工作区（讲义 PDF、notebook、报告）  
+- report\                # 各章报告（LaTeX 工程）
```

使用中文/空格路径会让部分 LaTeX/notebook 工具报错，**强烈建议全英文**。

2.8 常见问题与排错

现象	原因与解决
conda 不是内部或外部命令	未加 PATH；用 Anaconda Prompt，或重新安装时勾选 PATH
下载很慢	用清华镜像下载安装包；之后 conda/pip 也配镜像
conda env create 报错通道冲突	注释掉 .condarc 中自定义 channel 后重试；或改用 pip 安装
VS Code 找不到 scicomp 解释器	Ctrl+Shift+P → Python: Select Interpreter → 选 scicomp；若找不到，重启 VS Code 或 conda activate 后再开
Jupyter 找不到内核	在环境中执行 python -m ipykernel install --user --name scicomp
xelatex 未找到	尚未安装 TeX Live，见 00-05；临时可用 Overleaf/LoongTeX
中文输出乱码	终端用 chcp 65001 或 PowerShell；文件本身是 UTF-8 即可

本节小结与思考题

1. 为什么统一环境能减少一半的答疑时间？（提示：问题从“每个人机器不同”变为“版本差异”）
2. 在你自己电脑上跑通 check_env.py，把输出保存下来。
3. 如果机房已经预装好环境，你还需要做什么？（提示：至少验证 code、conda、xelatex 都在）

03 VS Code 使用与 Jupyter 工作流

本节目标：从现在起，你的“写代码”都发生在 VS Code 里。本节教你怎么打开项目、运行脚本、跑笔记本、查错误、用快捷键，让后面 8 章不被工具本身耽误。

本节目标

- 会用 VS Code 打开课程目录并在正确的 Python 环境中运行代码；
- 会新建与运行 .ipynb 笔记本：运行单元格、重启内核、导出；
- 会在终端激活 scicomp 环境、运行 pip/conda 命令；
- 会看“问题”面板与运行输出，能读懂简单报错；
- 记住最常用的 10 个快捷键。

3.1 打开文件夹与工作区

1. 启动 VS Code → 文件 → 打开文件夹（或 Ctrl+K Ctrl+O）；
2. 选择你的课程目录（例如 D:\Scientific_Computing_Class\course-dist）；
3. 左边资源管理器出现文件树；把 chapters/00-prep 等目录展开即可。

工作区 vs 文件夹：直接打开文件夹即可；只有需要多根目录时才用“工作区（.code-workspace）”。

3.2 解释器与内核（重要!）

- **解释器**（跑.py用）：Ctrl+Shift+P → Python: Select Interpreter → 选择 scicomp。
- **内核**（跑 notebook 用）：打开 .ipynb 后，右上角内核选择器同样选 scicomp；没有该内核时在终端执行：

```
conda activate scicomp
python -m ipykernel install --user --name scicomp
```

建议在项目根目录放一个 .vscode/settings.json 固定解释器（示例，路径按你的机器改）：

```
{
  "python.defaultInterpreterPath": "C:/Users/你的用户名/miniconda3/envs/scicomp/python.exe",
  "python.terminal.activateEnvironment": true,
  "jupyter.notebookFileRoot": "${workspaceFolder}"
}
```

3.3 运行 Python 脚本

1. 新建 hello.py，写：

```
print(" 你好, Python 科学计算")
print(2 ** 10)
```

2. 三种运行方式：

- 右上角的“运行”按钮（Play）；
- 右键 → 在终端中运行 Python 文件；
- 打开集成终端（Ctrl+`）手动执行 python hello.py 。

3. 注意终端中的当前目录（pwd / cd），文件路径建议按“相对项目根目录”写。

3.4 Jupyter 笔记本入门

新建与运行

- 文件 → 新建文件，保存为 lab0.ipynb（先保存才能识别为笔记本）；
- 每格支持 **代码单元格** 与 **Markdown 单元格**（用 M 切换模式，Y 切回代码）；
- 运行当前格：Shift+Enter（并跳到下一格）或 Ctrl+Enter（停在当前格）。

会用到的操作

操作	快捷键
上方插入单元格	A（命令模式下）
下方插入单元格	B
删除单元格	Ctrl+Shift+K 或 DD
保存	Ctrl+S
重启内核	Ctrl+Shift+P → Jupyter: Restart Kernel
全部运行	Run All（工具栏）
导出为 HTML/PDF	工具栏 → 导出（PDF 推荐用“导出为 HTML 再打印”或先装 LaTeX 依赖）

笔记本规范（从第 0 章养成）

1. 第一格用 Markdown 写标题、姓名、学号、日期；
2. 每个任务前用 Markdown 单元格写清“要做什么”；
3. 一个任务一个代码单元格，避免一个格子塞 200 行；
4. 输出重要的结果用 print 或直接显示，方便老师检查；
5. 运行顺序从上到下；中途改了前面的代码务必“重启内核并全部运行”再提交。

3.5 终端与包管理

- 在 VS Code 中按 'Ctrl+ ` ' 打开终端；
- 确认环境：

```
conda env list          # 看有哪些环境
conda activate scicomp  # 激活
where python            # 确认用的是 scicomp 的 python
```

- 装/卸包：

```
pip install 包名
pip uninstall 包名
conda install -c conda-forge 包名
```

注意：如果终端 python 还是系统 Python，说明没激活环境，请先 conda activate scicomp。

3.6 调试与排错

- **问题面板** (Ctrl+Shift+M)：显示语法/类型/导入错误；
- **运行与调试**：在行号左侧点击设置断点，按 F5 启动调试；F10 单步、F11 进入函数；
- **输出乱码**：终端输入 chcp 65001 切换 UTF-8；

• **报错读法**（自右向左读）：

Traceback (most recent call last):
 File "lab0.py", line 12, in <module>
 print(1/0)
ZeroDivisionError: division by zero

← 出错的调用链
← 文件与行号
← 最后一行才是真正的错误类型与信息

常用错误速查：NameError（名字没定义）、TypeError（类型不对）、IndexError（越界）、KeyError（字典缺键）、FileNotFoundError（路径不对）、ModuleNotFoundError（没装包/环境不对）。

3.7 常用快捷键表

功能	快捷键
快速打开文件	Ctrl+P
命令面板	Ctrl+Shift+P
切换终端	Ctrl+`` 运行单元格 Shift+Enter 注释/取消注释 Ctrl+/ 格式化文档 Shift+Alt+F 全局搜索 Ctrl+Shift+F 打开设置 Ctrl+, '

3.8 本节演示（跟着做一遍）

- 1. 新建 demo.ipynb;
- 2. Markdown 格写标题 “我的第一个课程笔记本”;
- 3. 代码格：

```
import sys
print(sys.executable)           # 看当前解释器
nums = [1, 2, 3, 4, 5]
print(" 平均值:", sum(nums) / len(nums))
```

- 4. 运行后，把 sys.executable 的输出和终端 where python 对比：应该都是 scicomp 的路径；
- 5. 交给老师/助教检查输出截图。

常见问题

问题	解决
右键没有 “在终端中运行”	确认安装了 Python 扩展并已选择解释器
笔记本运行报 “No kernel”	重启 VS Code，再选 scicomp；或执行上面 ipykernel 安装命令
中文终端乱码	chcp 65001；文件保存用 UTF-8
运行结果和老师不一样	先检查解释器；再检查每次是否 “重启内核并全部运行”

本节小结

- 先选对解释器/内核，再写代码；这两步解决 70% 的 “我电脑报错”。
- 笔记本 = 代码 + Markdown 说明；按 “一个任务一个单元格” 组织。
- 报错不可怕：最后一行 = 错误类型，往上找文件与行号。

思考题

1. `sys.executable` 输出的路径是 conda 的还是系统的？为什么这很重要？
2. 如果你在 VS Code 里运行 `.py` 成功，但运行 `.ipynb` 失败，最可能的原因是什么？

04 Python 编程快速强化

本节目标：你已学过语言程序设计课，本节不系统讲语法，而是用“自测 + 查漏”的方式强化科学计算最常用的 Python 能力：容器、函数、异常、文件、常用内置函数。先做自测，再针对性看讲解。

本节目标

- 能解释并操作 list/dict/tuple/set 的常见方法与切片；
- 熟练使用列表推导式、zip、enumerate、sorted、max/min/sum；
- 理解函数参数（默认值、*args、**kwargs）、作用域与返回值；
- 会写 try/except、with open(...) 读写文件；
- 养成命名、注释、if __name__ == "__main__" 的工程习惯。

4.1 课前自测（10 分钟，先不看答案）

在 exercises/quiz.ipynb 或纸上快速回答：

1. [1,2,3,4,5][::-1] 的结果？
2. {i: i*i for i in range(4)} 的结果？
3. sorted(["b","a","c"], reverse=True) 的结果？
4. zip([1,2],[3,4]) 转成 list 是什么？
5. 定义 def f(a=[]): ... 有什么风险？
6. try/finally vs try/except 的区别？
7. with open("a.txt") as f: 比 open/close 好在哪？
8. *args 与 **kwargs 分别装什么？
9. "a,b,c".split(",") 结果？
10. is 与 == 的区别（举例）？

答案与详解在 exercises/answers.ipynb。错 3 题以上，建议把本节的“查漏表”逐条过一遍。

4.2 变量与类型：会“拆开”就够了

类型	常用操作	注意
int / float	算术、//、**	浮点比较用 abs(a-b) < 1e-9
str	format、f-string、split/join	字符串不可变
list	追加、切片、推导式	可变；+ 是新建
tuple	解包、做字典键	不可变
dict	键值、get、遍历	键必须可哈希
set	去重、交并差	无序

一行代码示例：

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]
print(nums[1:5])           # [1, 4, 1, 5]
print(nums[::-1])          # 反转
print(sum(nums), max(nums), min(nums))
```

```
print(sorted(set(nums)))    # 去重排序
print({x: x * x for x in nums[:5]})
```

4.3 容器查漏表

切片: [start:stop:step], 含头不含尾; step 为负可倒序; [::-1] 最常用。

列表推导式: [表达式 for 项 in 可迭代 if 条件]; 不要嵌套太深, 超过两层就写循环。

字典方法:

```
d = {"a": 1, "b": 2}
print(d.get("c", 0))      # 0, 不会 KeyError
for k, v in d.items():    # 遍历键值
    print(k, v)
print(d.keys(), d.values())
```

set: set("hello") 去重; a & b、a | b、a - b。

4.4 流程控制与推导式

```
# 条件表达式 (三元)
label = " 及格" if score >= 60 else " 不及格"
```

```
# 列表推导 (科学计算里最常见的“批量变换”)
scores = [68, 91, 55, 76, 88]
passed = [s for s in scores if s >= 60]
```

```
# 循环里同时拿索引与值
for i, s in enumerate(scores):
    print(i, s)
```

```
# 两个列表打包
names = [" 张三", " 李四"]
for name, s in zip(names, scores[:2]):
    print(name, s)
```

4.5 函数: 默认值、不定长、作用域

```
def report(name, scores, *, title=" 成绩", fmt="%.1f"):
    """ 返回一行摘要字符串。 """
    total = sum(scores)
    avg = total / len(scores) if scores else 0.0
    return f"[{title}] {name}: 总分 {total}, 平均 {fmt % avg}"

print(report(" 张三", [80, 90]))
print(report(" 李四", [70, 75], title=" 期末"))
```

要点:

- 默认参数只在**定义时求值一次**: def f(a=[]) 会共享列表 → 改用 def f(a=None): a = [] if a is None else a;

- `*args` 收集位置参数为元组；`**kwargs` 收集关键字参数为字典；
- 函数内改全局变量要 `global`（尽量别用）；返回值用 `return`；
- 写 `docstring`（第一行字符串），是“会写代码”和“写得专业”的分水岭。

4.6 异常、文件与模块

异常处理

```
def safe_div(a, b):
    try:
        return a / b
    except ZeroDivisionError as e:
        print(" 除零啦:", e)
        return None
    finally:
        pass    # 无论成败都执行（比如关文件）
```

```
print(safe_div(10, 2))
print(safe_div(1, 0))
```

捕获要具体：不要裸 `except`；多个错误写 `except (ValueError, KeyError):`。

文件读写

```
# 写
with open("result.txt", "w", encoding="utf-8") as f:
    f.write(" 平均分 78.0\n")

# 读
with open("result.txt", encoding="utf-8") as f:
    lines = f.readlines()
print(lines)

# 读 csv (第 0 章只需 csv 模块；第 4 章再学 pandas)
import csv
with open("data/scores.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[:2])
```

模块与包

```
# 自己的模块
import math
from collections import Counter

print(math.pi)
print(Counter("hello world"))
```

- 自己写模块：同目录 `utils.py` 里定义函数，主程序 `from utils import xxx`；
- 主程序入口：`if __name__ == "__main__":`（被 `import` 时不执行、直接运行才执行）。

4.7 常用内置函数速查

函数	作用	例子
len/sum/max/min	长度/求和/最值	max(scores)
enumerate	带索引遍历	for i, x in enumerate(v):
zip	并行打包	dict(zip(names, scores))
sorted	返回新排序列表	sorted(d.items(), key=lambda kv: kv[1])
map/filter	映射/过滤（可用推导式替代）	list(map(str, nums))
any/all	存在/全部	any(x > 0 for x in nums)
round	四舍五入	round(3.14159, 2)
isinstance	类型判断	isinstance(x, (int, float))

4.8 工程习惯（现在养成，后面 8 章受益）

- 1. **命名**：变量 `score_list`、函数 `compute_average`；不用 `a1`、`data` 满天飞；
- 2. **注释**：解释“为什么”，不抄代码本身；
- 3. **一个文件只做一件事**；能拆函数就不堆在一个 `main` 里；
- 4. **导入放顶部**；第三方库按字母序排列；
- 5. **结果可复现**：随机数加种子（后面 NumPy 章会讲）；路径统一用相对路径；
- 6. **先跑通再优化**：不追求一行神代码。

4.9 本节综合小任务（交给 lab 的 Part B）

题目：读入 `lab/data/scores.csv`（字段：学号, 姓名,Python, 数学, 英语），用纯 Python 完成：

- 计算每个人的总分与平均分；
- 找出平均分最高与最低的学生；
- 把“总分 ≥ 240 ”的学生写入 `result.txt`；
- 统计 60 分以下的人数。

完整步骤见 `lab/lab.ipynb` 与 00-07 综合案例。

常见误区

误区	正确做法
<code>==</code> 比较小整数没问题，比较浮点数/字符串用 <code>is</code>	数值比较一律 <code>==</code> ； <code>is</code> 只用于 <code>None</code>
默认参数写 <code>=[]</code>	用 <code>=None</code> 再在函数内初始化
直接改列表时产生副作用	需要副本时用 <code>copy</code> 或切片 <code>[:]</code>
循环里 100 万次逐元素计算	后面学 NumPy 向量化（第 1 章核心）
<code>except</code> ：吞掉所有错误	捕获具体异常，至少 <code>print</code> 出来

本节小结与思考题

- 1. 用推导式写出：把 `scores` 中 60~90 分的小数保留两位并去掉重复。
- 2. 解释 `def add(a, b=10): return a + b` 的默认参数什么时候会被“记住”。
- 3. 为什么 `with open` 更安全？

05 LaTeX 基础与编译

本节目标：学会“把文字、公式、表格、图片排成一张漂亮的 PDF”。先了解在线平台（Overleaf、LoongTeX）作为备选，再掌握本地 VS Code + TeX Live 的主流流程，最后用课程模板写出你的第一份报告。

本节目标

- 知道 LaTeX 是什么、课程报告为什么推荐用它；
- 会用 **Overleaf / LoongTeX** 在线编译（网络/机房场景备选）；
- 会安装/检查 TeX Live，并配置 VS Code LaTeX Workshop 一键编译；
- 会写一个中文文档：标题、章节、数学公式、表格、插图；
- 会用 教学资源/LaTeX 模板/ 里的两个模板写作业/实验报告。

5.1 为什么课程用 LaTeX

优点	说明
排版专业	公式、表格、引用自动排版，论文/报告通用
稳定	同一个源码在任何电脑编译结果基本一致
可版本管理	.tex 是纯文本，可与 Git 配合
数学友好	科学计算报告充满公式，LaTeX 几乎是标准

代价：有一点点学习成本。**本章只教 4 件套**：文档骨架、公式、表格、插图 + 一键编译；其余按需查文档。

5.2 在线平台（备选，推荐至少注册一个）

Overleaf

- 网址：[链接](https://www.overleaf.com/)
- 优点：浏览器直接写、多人协作、内置大量模板（含中文模板）、无需安装；
- 缺点：中文支持需选 XeLaTeX 编译器并引入 ctex；免费版有编译时长/协作人数限制；国际网络/访问速度一般。

LoongTeX（龙文）

- 网址：[链接](https://www.loongtex.com/)
- 优点：国产、访问快、中文环境开箱即用（内置 ctex 模板）、文档/教程中文友好；
- 缺点：规模比 Overleaf 小、功能与长期稳定性需以官网为准。

在线 vs 本地

场景	建议
机房/公共机没装 TeX Live	用在线平台应急
在家、有网、机器带宽好	本地 TeX Live + VS Code（本节主流程）

场景	建议
小组协作写一份报告	Overleaf 协作更省事
课程期末报告 (要控排版)	本地模板更可控

注意: **在线平台也要选 XeLaTeX 编译器**, 否则中文会报错或乱码。

5.3 本地安装 TeX Live (Windows)

完整安装 (推荐, 一次到位)

1. 打开 [链接](https://www.tug.org/texlive/acquire-netinstall.html, 下载) install-tl-windows.exe (约 20 MB);
2. 运行安装程序: 选择 **完整安装** (约 7-8 GB, 包含 ctex、latexmk、xelatex 等); 网速慢可先在国内 CTAN 镜像下载 ISO 再安装;
3. 安装完成后验证:

```
xelatex --version
latexmk --version
```

精简安装 (硬盘小 / 网速受限)

- 安装时选择 scheme-small 或 scheme-basic, 缺哪个宏包再 tlmgr install 宏包名;
- **至少需要**: ctex、xecjk、amsmath、graphicx、hyperref、latexmk、xetex。

macOS / Linux

- macOS: brew install --cask mactex (或下载 MacTeX 安装包);
- Linux (含 CI): sudo apt install texlive-xetex texlive-lang-chinese texlive-latex-extra latexmk fonts-noto-cjk;
- 若安装不动, 直接用 5.2 的在线平台上课。

5.4 配置 VS Code + LaTeX Workshop

1. 安装扩展 **LaTeX Workshop** (james-yu.latex-workshop);
2. 打开设置 JSON (Ctrl+Shift+P → Open Settings (JSON)), 确认/添加:

```
{
  "latex-workshop.latex.recipes": [
    { "name": "xelatex", "tools": ["latexmk-xelatex"] }
  ],
  "latex-workshop.latex.tools": [
    {
      "name": "latexmk-xelatex",
      "command": "latexmk",
      "args": ["-xelatex", "-synctex=1", "-interaction=nonstopmode", "%DOC%"]
    }
  ],
  "latex-workshop.latex.autoBuild.run": "onSave",
```

```
"latex-workshop.view.pdf.viewer": "tab"
}
```

3. 在 .tex 文件首行写魔法注释: % !TeX program = xelatex;

4. 保存后应自动编译并在 VS Code 内预览 PDF; Ctrl+Alt+B 手动构建, Ctrl+Alt+V 看 PDF。

也可以直接用课程模板自带的 .vscode/settings.json (模板目录 latex_vscode_template/), 拷贝进你的项目即可。

5.5 第一份中文文档 (最小可编译示例)

```
% !TeX program = xelatex
\documentclass{ctexart}
\usepackage{amsmath}
\usepackage{graphicx}
\usepackage{hyperref}
```

```
\title{课程实验报告}
\author{张三\ 20230001}
\date{2026 年秋季}
```

```
\begin{document}
\maketitle
```

```
\section{实验目的}
学会用 XeLaTeX 编译中文文档。
```

```
\section{结果}
平均分公式:
\begin{equation}
\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i
\end{equation}
```

表格示例:

```
\begin{table}[h]
\centering
\begin{tabular}{|l|c|r|}
\hline
姓名 & Python & 数学 \\
\hline
张三 & 88 & 92 \\
李四 & 76 & 81 \\
\hline
\end{tabular}
\caption{成绩表}
\end{table}
```

图片示例:

```
\begin{figure}[h]
```

```
\centering
\includegraphics[width=0.6\textwidth]{example.png}
\caption{结果示意图}
\end{figure}

\end{document}
```

说明: ctexart 让文档支持中文; xelatex 负责字体; \begin{equation}...\end{equation} 是编号公式; tabular 是表格; includegraphics 插图。

5.6 编译方法与常见错误

编译命令

```
cd 报告目录
latexmk -xelatex main.tex          # 推荐: 自动多遍
# 或手动四连: xelatex main.tex && bibtex main && xelatex main.tex && xelatex main.tex
```

常见错误表

报错/警告	原因与解决
! Package ctex Error	没开 XeLaTeX (用了 pdflatex) → 改用 latexmk -xelatex
Missing character: There is no ...	字体缺字/空格命令错误; 中文必须 xelatex + ctex
! LaTeX Error: Filexxx.sty' not found 宏包未安装 →tlmgr install xxx !! Undefined control sequence 命令拼写错, 或忘了引宏包 (如 amsmath) !! Undefined references 需要多编译几遍; 用 latexmk 自动处理 图片显示不出 图片路径相对.tex 所在目录; 用\includegraphics 且文件存在 编译卡住/内存 关闭-synctex; 删掉.aux ' 后再编译	

5.7 使用课程模板

- **最简模板** latex_simple/: 一个 example.tex, 看标题/公式/列表/表格/结论;
- **完整模板** latex_vscode_template/: 含 references.bib 参考文献与 figures/ 示例图, 适合课程讲义/实验报告/期末大作业。

用法 (以作业报告为例):

1. 把 latex_vscode_template 复制为你的报告目录 (如 report/week0/)
2. 改 main.tex 标题、作者、日期
3. 写 section{实验目的} section{方法} section{结果} section{结论}
4. 插入公式/表格/图片 (见 5.5)
5. 保存 → 自动编译 → 导出 main.pdf 提交

注意: **不要手改模板里的宏包配置**, 先照抄; 想加宏包再按文档加 \usepackage.

5.8 在线平台的快速入门（备选）

- Overleaf: New Project → 选 ctexart 或任何中文模板 → 把编译器切到 XeLaTeX (Menu → Compiler) → 编辑左侧源码、右侧实时 PDF;
- LoongTeX: 注册 → 新建项目（默认中文模板）→ 粘贴 5.5 示例 → 编译即可；文档帮助见官网教程。

本节小结与思考题

1. 为什么中文必须 XeLaTeX? (提示：字体与编码)
2. 用课程模板写一个 5 行的小报告：标题、一段话、一个公式、一个表格，编译成 PDF。
3. 如果机房电脑装不了 TeX Live，你准备用哪条备选路径？

06 科学计算工作流与代码规范

本节目标：让第 0 章成为你“会做项目”的起点：目录规范、notebook 组织、代码风格、环境复现、Git 版本管理入门。

本节目标

- 建立全英文、按主题分层的项目目录；
- 按“一个任务一个单元格”组织笔记本并保证可复现；
- 知道科学计算代码的基本规范与命名；
- 用 `environment.yml` / `requirements.txt` 复现环境；
- 掌握 Git 的 4 个核心命令（选学，期末大作业前会用到）。

6.1 项目目录规范

```
course-dist/          # 你的工作区
+-- data/              # 原始数据（只读）
+-- src/ 或 scripts/   # 可复用 .py 脚本
+-- notebooks/         # 各章 .ipynb
+-- report/            # LaTeX 报告
+-- output/            # 生成结果（图/表/txt）
+-- environment.yml    # 环境定义
+-- README.md
```

- **路径全英文**：中文/空格路径会让 LaTeX、Jupyter、pandoc 等工具出幺蛾子；
- 数据与代码分离：数据放 `data/`，结果写 `output/`，不要混在源码目录。

6.2 Notebook 组织（重要）

1. **第一个 Markdown 单元格**：标题、姓名、学号、日期、依赖；
2. 按任务分块：## 任务 1 读取数据 → 代码 → 输出；## 任务 2 统计 → 代码 → 输出；
3. **从上往下能一键重跑**：提交前 Restart Kernel & Run All，确认没有“靠之前的变量”才能通过；
4. 中间过程用 Markdown 写“这里做了什么”，不要只有代码；
5. 大结果存文件（`output/result.txt`、`output/fig.png`），notebook 里只留摘要。

6.3 代码规范（够用版）

- 命名：函数动词开头（`compute_average`）、常量全大写、变量小写下划线；
- 一个函数一个职责；超过 20 行考虑拆分；
- 导入置顶：标准库、第三方、本地模块分组；
- 注释写“为什么”；不要在提交代码里留 `print(123)` 测试垃圾；
- 涉及随机结果一定要设种子（`random.seed` / 后续 `np.random.default_rng`）。

6.4 可复现环境

- 用课程提供的环境文件重建：`conda env create -f environment.yml`；
- 也可以用 pip 冻结：`pip freeze > requirements.txt`；

- 提交作业时注明 Python 版本与关键包版本 (python -V、pip list 截取)。

6.5 Git 入门 (选学, 期末前补)

安装 Git for Windows: [链接](https://git-scm.com/)。四个核心命令:)

```
git clone https://github.com/你的账号/仓库.git # 第一次
git add .
git commit -m "week0: 环境 + 报告"
git push
```

- .gitignore: 把 .venv、output/、*.pdf (如不需要提交) 排除;
- 只提交源码、笔记、报告源码; 不要提交大文件 (数据 > 50 MB 走网盘/云盘);
- 每完成一章提交一次, 方便回溯与协作。

6.6 提交与打包规范

第 0 章提交内容 (示例):

```
学号 _ 姓名 _ 第 0 章 _env_check.txt      # 环境自检输出
学号 _ 姓名 _ 第 0 章 _lab0.ipynb         # lab 完成版
学号 _ 姓名 _ 第 0 章 _lab0 报告.pdf      # LaTeX 报告
学号 _ 姓名 _ 第 0 章 _result.txt         # 数据统计结果
```

- 命名统一: 学号 _ 姓名 _ 第 N 章 _ 类型. 扩展名;
- PDF 用课程模板编译, 不交 .tex 可以不交 (老师按需收源码)。

6.7 本节小结

- 工具链稳定 = 项目能跑通的一半; 目录规范 = 能被复现的一半;
- Notebook 的 “可重跑” 比 “我运行过” 更重要;
- Git 不是期末考试, 但期末大作业小组协作时是 “保命” 技能。

思考题

1. 为什么数据、代码、输出要分开目录?
2. 什么是 “上一次运行留下的变量竟然被下一次用了” 的隐藏 bug? 如何避免?
3. 假如你 3 周后回来打开两周前的 notebook, 哪些做法能让你 5 分钟恢复上下文?

07 综合案例：从成绩单数据到 LaTeX 报告

本案例把第 0 章的三件事串成一条线：读 CSV 数据 → 用纯 Python 统计 → 画出简图 → 用 LaTeX 排版成课程报告。它也是期末大作业（数据分析 + 报告）的迷你演练。

本节目标

- 用 csv 模块读取课程成绩单；
- 用字典、列表、推导式、函数完成统计与排名；
- 把统计结果写入 result.txt；
- 用 matplotlib 画一张简单的成绩分布图（预告第 5 章，先用起来）；
- 用课程 LaTeX 模板把结果排成一份 1–2 页报告，编译出 PDF。

7.1 数据

数据文件：lab/data/scores.csv（20 名学生，字段：学号，姓名，Python，数学，英语）。

学号，姓名，Python，数学，英语
20230001, 张三, 88, 92, 79
20230002, 李四, 76, 81, 85
...

数据由 build/make_chap0_lab.py 生成，可复现（种子固定）。

7.2 任务 1：读取与清洗

```
import csv

with open("lab/data/scores.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

print(" 学生数:", len(rows))
print(rows[0])
# 检查是否有空值/非数字
bad = [r for r in rows if not r["Python"].strip() or not r["数学"].strip()]
print(" 异常行:", bad)
```

要求：把成绩转成 float；发现异常行时打印并跳过（本数据默认无异常，代码要能容错）。

7.3 任务 2：统计与排名

```
def compute(row):
    py, ma, en = float(row["Python"]), float(row["数学"]), float(row["英语"])
    total = py + ma + en
    return total, total / 3

for r in rows:
    r["总分"], r["平均分"] = compute(r)
```

```
by_avg = sorted(rows, key=lambda r: r[" 平均分"], reverse=True)
print(" 第一名:", by_avg[0][" 姓名"], by_avg[0][" 平均分"])
print(" 倒数第一:", by_avg[-1][" 姓名"], by_avg[-1][" 平均分"])

class_stats = {
    "Python 平均": sum(float(r["Python"]) for r in rows) / len(rows),
    " 平均分 ≥ 60 人数": sum(1 for r in rows if r[" 平均分"] >= 60),
    "90 分以上 (Python)": sum(1 for r in rows if float(r["Python"]) >= 90),
}
```

要求: 输出总分排名前 3 与最后 3; 统计各科平均分、最高分、最低分。

7.4 任务 3: 写出结果文件

```
with open("output/result.txt", "w", encoding="utf-8") as f:
    f.write("== 课程成绩统计 ==\n\n")
    for name, val in class_stats.items():
        f.write(f"{name}: {val:.2f}\n")
    f.write("\n排名前 3: \n")
    for r in by_avg[:3]:
        f.write(f" {r['姓名']} 平均 {r['平均分']:.1f}\n")
print(open("output/result.txt", encoding="utf-8").read())
```

注意: 先 `os.makedirs("output", exist_ok=True)`。

7.5 任务 4: 画一张简图 (预告第 5 章)

```
import matplotlib
matplotlib.use("Agg") # 无界面后端, 保存文件不弹窗 (上课/机房必备)
import matplotlib.pyplot as plt

names = [r[" 姓名"] for r in rows]
avgs = [r[" 平均分"] for r in rows]
plt.figure(figsize=(9, 4))
plt.bar(names, avgs)
plt.axhline(60, color="red", linestyle="--", label=" 及格线 60")
plt.xlabel(" 姓名"); plt.ylabel(" 平均分"); plt.title(" 课程平均分分布")
plt.legend()
plt.savefig("output/avg_scores.png", dpi=120)
plt.close()
print(" 已保存 output/avg_scores.png")
```

7.6 任务 5: LaTeX 报告 (核心交付物)

用 教学资源/LaTeX 模板/latex_vscode_template/ 复制成 report/week0/, 改 main.tex:

- 标题: 《第 0 章综合案例: 成绩单统计报告》, 作者 = 你的学号姓名;
- `\section{数据与方法}`: 说明数据来源 (20 人、3 门课) 与统计方法 (均值等);
- `\section{结果}`: 一个表格 (前 5 名), 一个公式 (平均分公式), 一张图 (avg_scores.png);
- `\section{结论}`: 写 2–3 句自己的发现;

- 编译: `latexmk -xelatex main.tex`, 得到 `main.pdf`。

表格示例:

```
\begin{tabular}{|l|c|c|}  
\hline  
姓名 & Python & 平均分 \\  
\hline  
张三 & 88 & 86.3 \\  
\hline  
\end{tabular}
```

7.7 任务 6: 自检清单

- ☐ 环境自检 `check_env.py` 全 OK;
- ☐ `python lab0.py` 能从头跑通 (无报错);
- ☐ `result.txt` 与 `avg_scores.png` 生成;
- ☐ LaTeX 报告编译出 PDF, 公式/表格/图都正常;
- ☐ 提交 4 个文件: `env_check.txt`、`lab0.ipynb` (或 `lab0.py`)、`result.txt`、报告 PDF。

7.8 拓展 (学完第 4 章后回看)

1. 用 `pandas` 重写统计 (一行 `df.mean()` 就能做完);
2. 用 `seaborn` 画更美观的分布图;
3. 把报告升级为 “每科通过率 + 班级分布直方图 + 简短分析”。

本节小结

这个案例虽然简单, 但走完了科学计算项目的**完整闭环**: 数据 → 编程统计 → 结果文件 → 可视化 → 排版报告。后面 8 章会在每一环上逐步替换成更专业的工具 (`NumPy/Pandas/Matplotlib/...`), 但 “闭环” 的思维方式从第 0 章就开始用了。

思考题

1. 为什么第 0 章用纯 Python 而不是直接上 `pandas`? (提示: 先懂原理, 再用工具)
2. 如果成绩表有 10 万行, 纯 Python 循环还够快吗? 第 1 章 `NumPy` 会怎么解决?